

Lernziele:

- Wiederholung und Verbesserung eines Pattern für Funktionen mit Parameter, Rückgabewert

Verwendete Architektur in dieser Session

Eine Befehlssatzarchitektur (englisch Instruction Set Architecture, kurz ISA) ist eine Schnittstelle zwischen Hardware und Software. Dabei werden zwei Aspekte beschrieben:

1. Welche Befehle unterstützt werden und wie diese im Maschinencode dargestellt werden.
2. Eine Assemblernotation für die Befehle.

Warum ist die ISA eine Schnittstelle zwischen Software und Hardware?

Die ISA beschreibt nur, welche Funktionen die Hardware unterstützen soll, jedoch nicht, wie dies konkret geschehen muss. Dieselbe ISA kann bezüglich unterschiedlicher Optimalitätskriterien (Leistung, Energieverbrauch, etc.) realisiert werden. Die konkrete Umsetzung einer ISA nennt man Mikroarchitektur. Eine bekannte ISA ist beispielsweise x86 mit den zugehörigen Mikroarchitekturen wie Intel i5, Intel i7, Intel i9, Intel Xeon und Intel Atom. Ein anderes Beispiel für eine ISA ist ARMv8.5-A mit den zugehörigen Mikroarchitekturen wie Apple M1 und Apple M2.

Für die Softwareentwicklung muss lediglich der passende Assembler-Code erzeugt oder von einem Compiler generiert werden. Wird Assemblercode für eine ISA in Maschinencode übersetzt, so läuft dieser auf jeder zugehörigen Mikroarchitektur. Programme müssen nur einmal für eine ISA übersetzt werden und nicht für jede einzelne Mikroarchitektur.

Eine Mikroarchitektur muss nicht zwangsläufig in Form von Hardware realisiert werden. Sie kann auch in Form von Software als virtuelle Maschine (im Prinzip ein Simulator) implementiert werden. Dies bietet Vorteile bei der Entwicklung neuer Hardware. Man spezifiziert zunächst einen neuen Befehlssatz und kann dann gleichzeitig eine Mikroarchitektur entwickeln und Software dafür schreiben.

In dieser Übung verwenden wir die Architektur, die in Session 10 verwendet wurde. Zur Erinnerung: In Session 10 wurde die ISA `ulm-ice40.isa` erweitert, um Funktionsaufrufe zu vereinfachen. Mit dem Kommando

```
ulm-generator --fetch ulm-ice40.isa
```

wurde die bisherige ISA-Beschreibung `ulm-ice40.isa` ins aktuelle Verzeichnis geladen. Dann wurde der Befehlssatz um zwei Befehle erweitert, indem folgende Zeilen hinzugefügt wurden:

```
0x0E    RR
:  call    %x
        ulm_absJump(ulm_regVal(x), x);

0x0F    U20_R
:  call    imm, %dest
        ulm_absJump(imm, dest);
```

Eine Mikroarchitektur in Form einer virtuellen Maschine und ein Assembler für diese ISA wurden dann mit

```
ulm-generator --install ulm-ice40.isa
```

gebaut.

Schritt 1 (Verbesserung des Pattern für Funktionen in Session 14)

In Session 14 haben wir ein Pattern entwickelt, um Funktionen mit Parametern und einem Rückgabewert zu realisieren. Von zentraler Bedeutung für das Pattern war die Verwendung eines Stacks. Dieser wurde für folgende Zwecke verwendet:

- Hinterlegen einer Rücksprungsadresse
- Übergabe von Parametern an die Funktionen
- Hinterlassen des Rückgabewertes

Der Zweck eines Patterns ist es, diese als Bausteine verwenden zu können, ohne sich jedes Mal Gedanken über die Details machen zu müssen, die in diesem Pattern versteckt sind. Bevor das Pattern erweitert wird solltet ihr euch nochmals mit den wichtigsten Merkmalen vertraut machen.

Struktur eines Programms mit Funktionen

Die Abbildung rechts zeigt die Struktur eines Programms mit zwei Funktionen, `foo` und `main`. Was man sich merken muss, ist folgendes:

1. Register `%1` hat jetzt eine Sonderbedeutung und darf nur noch verwendet werden, wenn eine Pattern-Regel dies vorsieht. In diesem Fall wird man `%SP` statt `%1` schreiben.
2. Das Text-Segment sollte mit einem **Start-Block** beginnen.
3. Jede Funktion beginnt mit einem Label, das dem Funktionsnamen entspricht. Diesem Label folgt ein **Prologue-Block**. Die Funktion endet mit einem **Epilogue-Block**. Dazwischen kann die Funktion implementiert werden.
4. Die Ausführung des Programms beginnt "im Prinzip" mit der ersten Instruktion innerhalb des Funktionskörpers von `main`.

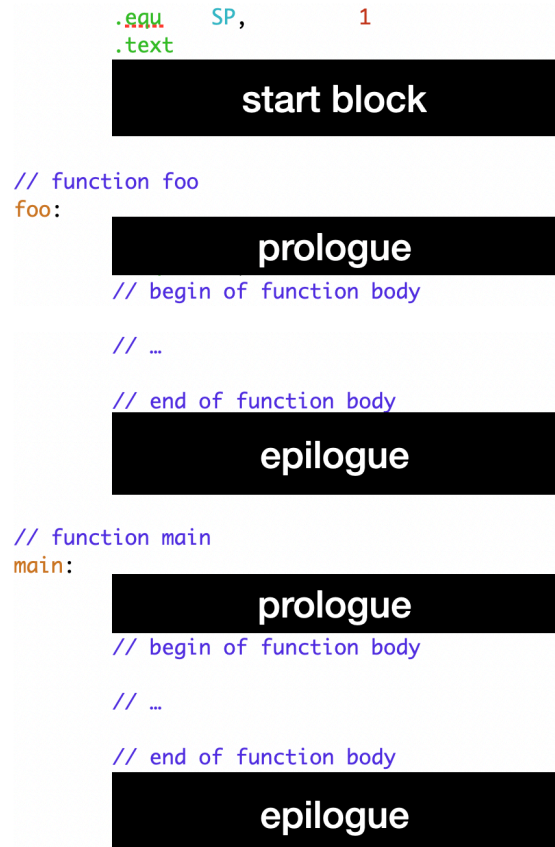
Die genannten Blöcke sind absichtlich als Black-Boxes dargestellt. Man kann sie mittels blindem Copy&Paste einfügen, ohne zu wissen, dass diese Blöcke jeweils folgenden Zweck haben:

- Mit dem Start-Block wird der Stack initialisiert und dann die Funktion `main` aufgerufen. Der Rückgabewert von `main` wird dann als Exit-Code für die Halt-Instruktion verwendet.
- Im **Prologue** wird die Rücksprungsadresse auf den Stack gelegt.
- Im **Epilogue** wird die Rücksprungsadresse vom Stack genommen und zum Aufrufer zurückgesprungen.

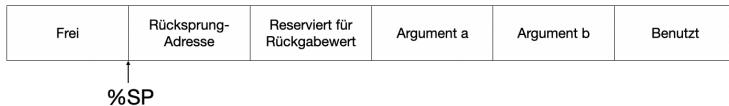
Callee-Pattern: Zugriff auf Parameter und hinterlegen eines Rückgabewertes

Eine Funktion erwartet eine gewisse Anzahl von Parametern und kann einen Rückgabewert hinterlassen. In höheren Programmiersprachen wird durch die Deklaration die Anzahl der Parameter festgelegt und ob die Funktion etwas zurückgibt. Außerdem wird durch die Deklaration der jeweilige Typ eines Parameters und des Rückgabewerts beschrieben. Diesen Luxus, dass der Code bereits eine Art von Dokumentation enthält, hat man bei der Assembler-Programmierung nicht. Außerdem hat man kein Tool, das einem bei der Buchführung hilft und einen zum Beispiel freundlich daran erinnert, "der dritte Parameter ist gar kein Zeiger, den solltest du nicht dereferenzieren".

In unserem Pattern gehen wir davon aus, dass für jeden Parameter sowie für den Rückgabewert immer 8 Byte verwendet werden. Wir müssen uns also "nur" merken, wie viele Parameter es gibt und welchen Typ ein Parameter hat (Character, Unsigned-Integer, Signed-Integer, Zeiger, etc.).



Das Pattern geht bei einer Funktion mit zwei Parametern davon aus, dass diese wie folgt relativ zum Stackpointer `%SP` liegen:



Hier war das Pattern in Session 14 noch nicht "hübsch", denn man soll ja eben nicht jedes Mal dieses Bildchen zeichnen müssen, um zu wissen, mit welchem Offset bezüglich des Stackpointers auf Parameter zugegriffen werden muss! Man könnte das besser verstecken, indem man folgende Equ-Direktiven am Anfang des Assembler-Programms verwendet (und gegebenenfalls erweitert):

```
.equ    retval, 8
.equ    param0, 16
.equ    param1, param0 + 8
.equ    param2, param1 + 8
```

Damit erhält man folgendes Pattern, um abhängig vom Typ Parameter zu laden und einen Rückgabewert zu hinterlegen:

Datentyp	Parameter 0 in %2 laden	Parameter 1 in %2 laden	%2 als Rückgabewert hinterlegen
8 Byte	movq param0(%SP), %2	movq param1(%SP), %2	movq %2, retval(%SP)
1 Byte unsigned	movzbq param0(%SP), %2	movzbq param1(%SP), %2	movb %2, retval(%SP)
1 Byte signed	movsbq param0(%SP), %2	movsbq param1(%SP), %2	movb %2, retval(%SP)

Caller-Pattern: Hinterlegen von Argumenten und Abholen eines Rückgabewertes

Das Pattern für die Übergabe war ebenfalls noch sehr technisch. Der Caller macht Platz auf dem Stack für die Argumente und den Rückgabewert (aber nicht für die Rücksprungsadresse, denn die zu speichern ist das Problem des Callee), hinterlegt die Argumente, ruft die Funktion auf und holt sich anschließend den Rückgabewert.

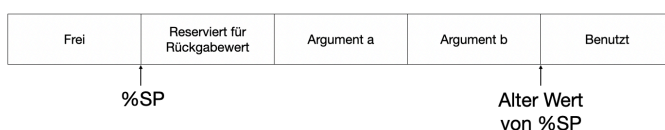
In folgendem Beispiel wird eine Funktion `sum` mit zwei Argumenten aufgerufen. Zunächst wird auf dem Stack Platz für 24 Bytes geschaffen (16 für die Argumente, 8 für den Rückgabewert) und die Argumente in den Stack geschrieben:

```

subq    3*8,          %SP,          %SP
loadq   2,            %2
movq    %2,           8(%SP)
loadq   3,            %2
movq    %2,           16(%SP)
call    sum,          %2
movq    (%SP),         %3
addq    3*8,          %SP,          %SP

```

Wenn die Call-Instruktion erreicht wird, kann man den Stack wie folgt beschreiben, wobei Argument a den Wert 2 und Argument b den Wert 3 hat:



Anschließend wird der Rückgabewert vom Stack geladen und der Stackpointer wieder auf den ursprünglichen Wert erhöht.

Hier ist zunächst störend, dass der Caller andere Offsets verwenden muss, um auf die Argumente und den Rückgabewert zugreifen zu können. Man muss hier also formal zwischen Argumenten (was der Caller übergibt) und Parametern (was der Callee erwartet) unterscheiden. Etwas

aufhübschen kann man das Ganze, indem man folgende Equ-Direktiven am Anfang des Assembler-Programms verwendet, um Offsets für die Argumente zu vereinbaren:

```
.equ    arg0, 8
.equ    arg1, arg0 + 8
.equ    arg2, arg1 + 8
```

Damit ist der Funktionsaufruf von `sum` zumindest etwas einfacher zu lesen:

```
subq    3*8,      %SP,      %SP
loadz   2,        %2
movq    %2,       arg0(%SP)
loadz   3,        %2
movq    %2,       arg1(%SP)
call    sum,      %2
movq    (%SP),    %3
addq    3*8,      %SP,      %SP
```

Aufgabe

In Session 14 habt ihr folgendes Programm in Assembler umgeschrieben. Wendet darauf die verbesserten Patterns an:

```
@ <stdio.h>

fn readInt(): u64
{
    local val: u64 = 0;
    local ch: int;

    while ((ch = getchar()) != EOF) {
        if (ch < '0' || ch > '9') {
            break;
        }
        val = val * 10 + ch - '0';
    }
    return val;
}

fn msg(str: -> char) {
    local ch: char;

    while ((ch = *str) != 0) {
        putchar(ch);
        ++str;
    }
}

fn main(): u64
{
    msg("Enter a number: ");
    local val: u64 = readInt();
    msg("Use echo $? and have a nice day!\n");
    return val;
}
```

Schritt 2 (Pattern mit Stackpointer und Framepointer)

Auch beim Design von Software-Patterns gibt es eine Art von Energieerhaltungssatz. Wenn man ein Pattern so verbessert, dass es einfacher und flexibler zu benutzen ist, dann müssen die versteckten Details des Patterns entsprechend komplexer sein. In diesem Schritt wird zunächst die Verwendung des verbesserten Patterns vorgestellt, ohne genau darauf einzugehen, welche schmutzigen Details versteckt werden.

Reservierte Register

Ab jetzt sind die Register `%1` und `%2` reserviert. Es werden folgende Equ-Direktiven am Anfang des Assembler-Programms vereinbart:

```
.equ    SP,    1
.equ    FP,    2
```

Register `%SP` ist nach wie vor der Stackpointer und Register `%FP` werden wir als sogenannten Framepointer bezeichnen (was immer das bedeuten mag).

Weitere Equ-Direktiven

```
.equ    retval, 16
.equ    param0, 24
.equ    param1, param0 + 8
.equ    param2, param1 + 8
```

Start-Block

```
# call: fn main(void): i64
subq    8*(3 + 0),    %SP,    %SP
call    main,        %3
movq    retval(%SP), %3
addq    8*(3 + 0),    %SP,    %SP
halt    %3
```

Prologue

```
movq    %3,          (%SP)
movq    %FP,          8(%SP)
movq    %SP,          %FP
// begin function body
```

Epilogue

```
// end function body
movq    %FP,          %SP
movq    8(%SP),        %FP
movq    (%SP),          %3
ret     %3
```

Caller: Funktionsaufruf

- Bei `n` Argumenten wird auf dem Stack `8*(3 + n)` Bytes Platz geschaffen. Also mindestens 24 Bytes und pro Argument zusätzlich 8 Bytes.
- Argumente werden relativ zum Stackpointer `%SP` in den Stack gelegt und auch der Rückgabewert wird relativ zum Stackpointer vom Stack geladen.

Datentyp	%3 als Argument 0 übergeben	%3 als Argument 1 übergeben	Rückgabewert in %3 laden
8 Byte	<code>movq %3, param0(%SP)</code>	<code>movq %3, param1(%SP)</code>	<code>movq retval(%SP), %3</code>
1 Byte unsigned	- " -	- " -	<code>movzbq retval(%SP), %3</code>
1 Byte signed	- " -	- " -	<code>movsbq retval(%SP), %3</code>

- Die Rücksprungadresse liegt nach dem Funktionsaufruf im Register `%3`.

Hier wird beispielsweise die Funktion `sum` mit den Argumenten `1` und `2` aufgerufen und der Rückgabewert nach dem Funktionsaufruf im Register `%3` geladen:

```
subq    8*(3 + 2),    %SP,    %SP
loadz   1,            %3
movq    %3,            param0(%SP)
loadz   2,            %3
movq    %3,            param1(%SP)
call    sum,          %3
movq    retval(%SP),  %3
addq    8*(3 + 2),    %SP,    %SP
```

Callee: Zugriff auf Parameter und Rückgabewert hinterlegen

Der Callee verwendet die gleichen Offsets wie der Caller. Allerdings verwendet der Callee diese relativ zum Framepointer **%FP**:

Datentyp	Parameter 0 in %3 laden	Parameter 1 in %3 laden	%3 als Rückgabewert hinterlegen
8 Byte	<code>movq param0(%FP), %3</code>	<code>movq param1(%FP), %3</code>	<code>movq %3, retval(%FP)</code>
1 Byte unsigned	<code>movzbq param0(%FP), %3</code>	<code>movzbq param1(%FP), %3</code>	<code>movb %3, retval(%FP)</code>
1 Byte signed	<code>movsbq param0(%FP), %3</code>	<code>movsbq param1(%FP), %3</code>	<code>movb %3, retval(%FP)</code>

Beispiel:

Das Programm:

```
fn sum(a: u64, b: u64): u64
{
    return a + b;
}

fn main(): u64
{
    return sum(2, 3);
}
```

kann man damit wie folgt in Assembler umschreiben:

```

1      .equ    SP,      1
2      .equ    FP,      2
3
4      .equ    retval, 16
5      .equ    param0, 24
6      .equ    param1, param0 + 8
7      .equ    param2, param1 + 8
8
9      loadz   0,                %SP
10
11     # call: fn main(void): i64
12     subq    8*(3 + 0),         %SP,    %SP
13     call    main,             %3
14     movq    retval(%SP),       %3
15     addq    8*(3 + 0),         %SP,    %SP
16     halt    %3
17
18
19 main:
20     movq    %3,                (%SP)
21     movq    %FP,               8(%SP)
22     movq    %SP,              %FP
23     // begin function body
24
25     subq    8*(3 + 2),         %SP,    %SP
26     loadz   2,                %3
27     movq    %3,               param0(%SP)
28     loadz   3,                %3
29     movq    %3,               param1(%SP)
30     call    sum,              %3
31
32     movq    retval(%SP),       %3
33     movq    %3,               retval(%FP)
34     addq    8*(3 + 2),        %SP,    %SP
35     // end function body
36     movq    %FP,              %SP
37     movq    8(%SP),           %FP
38     movq    (%SP),            %3
39     ret     %3
40
41 sum:
42     movq    %3,                (%SP)
43     movq    %FP,               8(%SP)
44     movq    %SP,              %FP
45     // begin function body
46
47     movq    param0(%FP),       %3
48     movq    param1(%FP),      %4
49     addq    %3,                %4,    %3
50     movq    %3,               retval(%FP)
51     // end function body
52     movq    %FP,              %SP
53     movq    8(%SP),           %FP
54     movq    (%SP),            %3
55     ret     %3
56
```

Aufgabe: Passt das Assembler-Programm aus Schritt 1 entsprechend an.